

Università degli Studi di Milano

FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA

GIOVANNI DEGLI ANTONI



CORSO DI LAUREA TRIENNALE IN
INFORMATICA

IMPLEMENTAZIONE DIGITALE DI UN GIOCO DA
TAVOLO CON GRAFICA INTERATTIVA PER IL
POTENZIAMENTO DELLE COMPETENZE SOCIALI DI
ADOLESCENTI AUTISTICI

Relatore: Prof. Nunzio Alberto Borghese

Elaborato Finale di:
Samuel De Benedictis
Matr. Nr. 855264

ANNO ACCADEMICO 2025-2026

Indice

Indice	i
1 Introduzione	1
1.1 L'autismo	1
1.2 Il CFPIL di Varese	2
1.3 Competenze sociali negli adolescenti autistici	2
1.4 Il gioco da tavolo	3
1.5 <i>La Città degli Imprevisti</i>	4
2 Stato dell'arte	6
2.1 Approcci al social skills training nell'ASD	6
2.2 Il gioco come strumento di intervento	6
2.3 Serious games digitali: stato delle evidenze	7
2.4 Posizionamento del presente lavoro	7
3 Tecnologie utilizzate	8
3.1 Framework e librerie di sviluppo	8
3.1.1 Next.js	8
3.1.2 TypeScript	8
3.1.3 React	9
3.1.4 Zustand	9
3.1.5 Tailwind CSS	9
3.2 Strumenti di qualità del codice	9
3.2.1 Biome	9
3.2.2 Husky e Commitlint	9
3.3 Strumenti di testing e documentazione	10
3.3.1 Vitest	10
3.3.2 Playwright	10
3.3.3 Storybook	10

4	Implementazione	11
4.1	Contesto d'uso e scelte progettuali	11
4.2	Architettura generale	11
4.3	Struttura ad alto livello	12
4.4	Specifiche funzionali	12
4.4.1	Configurazione della partita	13
4.4.2	Svolgimento del turno	14
4.4.3	Salvataggio e ripristino della partita	14
4.5	Descrizione funzionale	14
4.5.1	Schermata Home — Configurazione della partita	14
4.5.2	Schermata di gioco	15
4.5.3	Menu di navigazione	18
4.5.4	Schermata Istruzioni	18
4.5.5	Schermata Modalità avanzata	18
4.5.6	Schermata Ripristina partita	20
4.5.7	Schermata Feedback	20
4.6	Architettura dei moduli	21
4.6.1	Domain Model	22
4.6.2	State Management	22
4.6.3	Presentation Layer	23
4.7	Scelte implementative rilevanti	24
4.7.1	Il tipo <code>GameActionResult</code> e il Command Pattern	24
4.7.2	Gestione dello stato con Zustand e persistenza	25
4.8	Design pattern utilizzati	26
4.8.1	Command Pattern	26
4.8.2	Facade Pattern	26
4.8.3	Builder Pattern	27
4.8.4	Observer Pattern	27
4.8.5	Manager Pattern	27
4.8.6	Model-View-Controller	27
4.9	Modalità multi-dispositivo	28
4.9.1	Architettura delle sessioni	28
4.9.2	Connessione dei giocatori tramite QR code	29
4.9.3	Comunicazione in tempo reale tramite <i>long polling</i>	29
4.9.4	Rilevamento dell'inattività	29
4.9.5	Svolgimento del turno in modalità multi-dispositivo	30
5	Test	31
5.1	Raccolta di feedback	31

6 Conclusioni	32
Bibliografia	33

Capitolo 1

Introduzione

1.1 L'autismo

Il termine "*autistico*" nasce come aggettivo descrittivo: lo psichiatra svizzero Eugen Bleuler lo coniò per indicare la chiusura in sé stessi di alcuni suoi pazienti (dal greco *autos*). Solo in seguito fu adottato da altri autori per descrivere difficoltà simili osservate in bambini e adolescenti.

Già nel 1926 la psichiatra Grunya Efimovna Sukhareva aveva documentato con grande dettaglio le caratteristiche autistiche di sei bambini — eppure la diagnosi rimase quella di psicopatia schizoide. Ci vollero altri vent'anni prima che l'autismo venisse riconosciuto come condizione a sé.

Nel 1943, in modo indipendente, due ricercatori arrivarono a conclusioni simili. Hans Asperger, nella sua tesi di dottorato, descrisse la condizione di alcuni ragazzi basandosi sull'osservazione di circa duecento pazienti, usando il termine "*psicopatia autistica*". Nello stesso anno Leo Kanner riportò le osservazioni su undici bambini con "*marcate difficoltà nelle relazioni sociali e nella comunicazione*" e, in un lavoro successivo, propose il nome "*autismo infantile*".

Negli anni Settanta Lorna Wing mise in discussione l'idea che l'autismo potesse essere ricondotto a criteri diagnostici univoci: fornì le prime prove sistematiche della sua natura composita e introdusse l'espressione "*disturbo dello spettro autistico*" (ASD, dall'inglese *Autism Spectrum Disorder*).

Il termine "*spettro*" non è neutro. Come sottolinea Surian [1], riprende la concezione dimensionale già implicita nel lavoro di Asperger: la differenza tra persone con o senza autismo va pensata come una differenza di grado, non di qualità.

Nella pratica, la diagnosi viene posta quando una costellazione di tratti si manifesta nello stesso individuo in forma molto accentuata: propensione per la ripetizione di alcune attività, spiccata attenzione ai dettagli, difficoltà nell'accettare cambiamenti. È proprio su

uno di questi versanti — quello delle competenze sociali — che il presente lavoro intende intervenire.

1.2 Il CFPIL di Varese

Il *Centro Formazione Professionale Integrazione Lavorativa* (CFPIL), istituito dalla Provincia di Varese, rappresenta un punto di riferimento consolidato da oltre trent'anni per l'utenza con disabilità intellettiva, fisica e sensoriale. Nel 2002, il centro è stato integrato all'interno dell'"Agenzia Formativa della Provincia di Varese", divenuta successivamente Azienda Speciale nel dicembre 2009.

I progetti del CFPIL costituiscono una risposta strutturata alle esigenze specifiche di adolescenti, giovani e adulti con disabilità, offrendo percorsi formativi mirati e successivi inserimenti lavorativi.

L'attività formativa è progettata e monitorata attraverso équipe multidisciplinari composte da educatori professionali, psicologi e assistenti sociali. Questo approccio consente di predisporre curricula formativi personalizzati, alcuni dei quali si realizzano in contesti lavorativi all'interno di realtà produttive, aziende private ed enti pubblici.

Il CFPIL svolge una funzione di mediazione tra le persone con disabilità e il mondo del lavoro, erogando un sistema integrato di servizi che comprende:

- Informazione, valutazione e orientamento professionale;
- Formazione professionale specializzata;
- Integrazione lavorativa;
- Monitoraggio post-assunzione.

Il centro promuove relazioni sistemiche con il tessuto sociale, scolastico, produttivo e istituzionale del territorio, al fine di definire e realizzare progetti formativi condivisi. Attraverso questa rete collaborativa, il CFPIL intende contribuire alla diffusione del principio della diversità come risorsa individuale e allo sviluppo di una "cultura della diversità" fondata sul rispetto e sull'inclusione. [2]

1.3 Competenze sociali negli adolescenti autistici

Il potenziamento delle competenze sociali degli utenti del CFPIL costituisce uno degli obiettivi primari dell'istituto, riconosciuto come condizione imprescindibile per favorire l'inserimento lavorativo e la partecipazione attiva alla vita comunitaria.

Per molti adolescenti con disturbo dello spettro autistico le competenze sociali rappresentano un'area di significativa difficoltà. Con questo termine si fa riferimento alla

capacità di interagire in modo adeguato al contesto, di comunicare bisogni e intenzioni, di rispettare turni e ruoli e di gestire situazioni di conflitto: abilità che risultano spesso più determinanti delle competenze tecniche ai fini dell'integrazione lavorativa.

Sul piano neuropsicologico, queste difficoltà derivano da una combinazione di fattori tra loro correlati: bassa motivazione spontanea a iniziare interazioni sociali; tendenza a interazioni routinizzate anziché reciproche; difficoltà nella lettura di regole sociali implicite, come il linguaggio del corpo, il tono della voce o i segnali di disinteresse dell'interlocutore; deficit di teoria della mente, ovvero la capacità di inferire pensieri e intenzioni altrui [3]. Quest'ultimo deficit può portare, ad esempio, a prolungare una conversazione su un argomento di interesse personale senza percepire il disinteresse del proprio interlocutore, o a non cogliere il significato di un commento sarcastico.

Il CFPIL persegue questo obiettivo principalmente attraverso laboratori pratici a carattere professionalizzante, nei quali l'attività manuale diventa il mezzo attraverso cui esercitare e consolidare abilità relazionali in contesti strutturati e realistici, con ruoli definiti e procedure sequenziali che riproducono le dinamiche del mondo del lavoro.

Lavorare in contesti reali e motivanti, con una logica procedurale, permette di sviluppare le competenze sociali in modo più efficace rispetto agli esercizi astratti — e favorisce la generalizzazione di quanto appreso al di fuori del contesto scolastico o terapeutico. A questo si aggiunge un dato non scontato: gli adolescenti autistici esprimono interesse per la socializzazione, e il gioco è uno dei contesti in cui questo interesse emerge con più facilità [4]. Il formato ludico diventa quindi un canale di intervento, non un ripiego.

1.4 Il gioco da tavolo

Per i bambini e gli adolescenti con disturbo dello spettro autistico, il gioco riveste un'importanza terapeutica significativa: rappresenta una cornice educativa motivante in cui sperimentare abilità, consolidare comportamenti e relazionarsi con le persone circostanti. Qualsiasi gioco, indipendentemente dal suo grado di serietà, insegna implicitamente ai partecipanti come comportarsi in un contesto di gruppo; i meccanismi di avanzamento e ricompensa integrati lo rendono inoltre particolarmente motivante rispetto ad altre forme di intervento educativo [5]. Tra le varie forme di gioco, quella che raggiunge il più alto livello di strutturazione è il gioco da tavolo, caratterizzato dall'utilizzo di elementi fisici definiti (pedine, carte, tabelloni, dadi) e da un insieme di regole esplicite che delimitano uno spazio e un tempo di gioco ben determinati. Questa struttura chiusa e ben delimitata lo rende particolarmente adatto al contesto autistico. Le analogie con le attività di lavoro indipendente previste dalla metodologia *Treatment and Education of Autistic and related Communication handicapped Children* (TEACCH) non sono casuali: entrambe si svolgono seduti al tavolo, sono strutturate nel tempo e nello spazio, hanno una fine prestabilita e

privilegiano il codice visivo rispetto a quello verbale [6, 3]. Non a caso, tra le attività raccomandate dall'approccio TEACCH per il lavoro sulle competenze sociali di adolescenti figurano esplicitamente giochi da tavolo in gruppo e videogiochi [3].

A differenza delle attività di lavoro individuale, tuttavia, il gioco da tavolo richiede la partecipazione di almeno due soggetti: si configura quindi come un'interazione focalizzata faccia a faccia, offrendo opportunità di interazione sociale più ricche rispetto alle alternative esclusivamente digitali [5]. Anche nella versione digitale oggetto del presente elaborato, il gioco rimane un'esperienza condivisa che si svolge fisicamente attorno a uno schermo comune.

Una revisione sistematica della letteratura ha riscontrato esiti positivi in tutti gli studi analizzati sull'uso dei giochi come strumento di *social skills training* per persone autistiche [4]. La ricerca in questo ambito ha privilegiato strumenti digitali sviluppati appositamente — ambienti virtuali, robot, agenti conversazionali — mentre le implementazioni digitali di giochi da tavolo strutturati rimangono un approccio ancora poco esplorato.

L'implementazione digitale descritta in questo elaborato si colloca in questo spazio. Rispetto alla versione fisica, il formato digitale consente di adattare parametri come il numero di caselle, la tipologia delle sfide o il numero di giocatori direttamente dall'interfaccia — senza dover riprogettare e ristampare i componenti. L'interazione, però, rimane quella del gioco da tavolo: faccia a faccia, attorno a uno schermo condiviso.

1.5 *La Città degli Imprevisti*

La Città degli Imprevisti nasce da una collaborazione diretta tra utenti e docenti del CFPIL di Varese, che hanno progettato insieme un gioco centrato sulle competenze sociali. La struttura di base è quella del Gioco dell'Oca — un percorso lineare, un dado, le pedine — arricchita con caselle speciali che attivano minigiochi a carattere sociale ed emotivo. L'obiettivo è arrivare per primi all'ultima casella, ma lungo il percorso i giocatori si trovano a dover comunicare, collaborare, riconoscere emozioni — non come esercizio isolato, ma come parte del gioco stesso.

Le caselle speciali costituiscono il cuore educativo del gioco e sono progettate per sollecitare competenze diverse:

- **Mimo** — Il giocatore deve mimare una parola o una frase; allena la comunicazione non verbale.
- **Quiz** — Viene posta una domanda a risposta aperta o a scelta multipla; stimola l'attenzione e il ragionamento.
- **Parole alle spalle** — Il giocatore traccia con il dito una parola sulla schiena di un compagno, che deve riconoscerla; promuove il contatto fisico guidato e la comunicazione tattile.

- **Disegno dettato** — Un giocatore descrive a parole un'immagine che un compagno deve riprodurre graficamente; esercita la comunicazione verbale precisa e la collaborazione.
- **Emozioni in musica** — Viene proposta un'emozione e il giocatore deve scegliere una canzone che la rappresenti; allena l'espressione emotiva.
- **Indovina l'emozione** — Viene mostrato un volto e il giocatore deve identificare l'emozione espressa; allena il riconoscimento delle emozioni attraverso il volto.
- **Cosa faresti se...** — Il giocatore risponde a uno scenario ipotetico e gli altri valutano la risposta; allena la prospettiva altrui, una delle abilità più difficili da sviluppare nel ASD.
- **Test fisico** — Viene proposta una prova motoria da eseguire; favorisce la coordinazione e la gestione del corpo nello spazio condiviso.
- **Battaglia** — Quando due giocatori si trovano sulla stessa casella devono sfidarsi a morra cinese (carta-forbice-sasso); introduce la competizione regolamentata e la gestione dell'esito (vittoria e sconfitta).
- **Caselle movimento** — Alcune caselle fanno avanzare o retrocedere il giocatore; replicano la meccanica classica del Gioco dell'Oca, introducendo l'elemento della fortuna e dell'attesa.

Alcune di queste tipologie trovano riscontro nella letteratura empirica: come riportato da Atherton e Cross [5], giochi di riconoscimento delle espressioni facciali hanno prodotto miglioramenti significativi in bambini con disturbo dello spettro autistico (Tanaka et al., 2010); attività di mimica con contenuto emotivo hanno dimostrato di allenare la decodifica del linguaggio non verbale (Dell'Angela et al., 2020); il gioco di ruolo da tavolo con adolescenti autistici ha prodotto miglioramenti nel benessere emotivo e nelle amicizie (Katō, 2019).

Le caselle speciali non sono scelte in modo arbitrario: nel loro insieme, coprono le aree di difficoltà strutturale che la letteratura individua come caratteristiche del ASD — lettura delle espressioni facciali e del linguaggio non verbale, decodifica dei gesti, assunzione della prospettiva altrui, comunicazione verbale precisa, gestione delle emozioni, collaborazione e competizione regolamentata [3]. La varietà delle sfide garantisce che durante la partita vengano esercitate competenze eterogenee, mantenendo alto il coinvolgimento dei partecipanti.

Capitolo 2

Stato dell'arte

2.1 Approcci al social skills training nell'ASD

Il *social skills training* (SST) per persone con disturbo dello spettro autistico comprende un insieme eterogeneo di approcci: dall'istruzione individuale basata su tecniche di modifica del comportamento, ai programmi di gruppo strutturati, fino agli interventi mediati da pari e alle metodologie basate sulla routine visiva, come TEACCH [3]. Nonostante la varietà degli approcci e il numero crescente di studi empirici, non si è ancora affermata una pratica ampiamente condivisa [7]: i risultati tendono a essere positivi nel contesto di intervento, ma faticano a generalizzarsi ad altri ambienti. Una variabile critica trasversale a tutti gli approcci è la motivazione del partecipante: perché un intervento funzioni, il soggetto deve trovare nelle attività proposte uno stimolo sufficiente a volersi impegnare [3].

2.2 Il gioco come strumento di intervento

Negli ultimi anni il gioco ha acquisito centralità come canale di intervento per le competenze sociali nelle persone con ASD. Atherton e Cross [5] hanno documentato una vasta gamma di applicazioni — dai giochi analogici a quelli digitali — evidenziando come il formato ludico offra un contesto motivante in cui comportamenti sociali complessi possono essere esercitati in modo naturale, grazie ai meccanismi di avanzamento e ricompensa integrati nella struttura del gioco. Walsh, Linehan e Ryan [4] hanno analizzato specificamente l'uso del gioco come strumento di SST per bambini e adolescenti autistici: tutti gli studi inclusi nella revisione riportano esiti positivi, sebbene gli autori segnalino la necessità di un approccio teoricamente più fondato nella definizione degli obiettivi comportamentali. Un dato rilevante emerso dalla revisione è che tutti gli studi includevano una

componente tecnologica, a conferma di come la ricerca si sia orientata prevalentemente verso strumenti digitali.

2.3 Serious games digitali: stato delle evidenze

Nell'ambito degli strumenti digitali, i *serious games* — giochi il cui scopo primario non è l'intrattenimento ma la trasmissione di conoscenze o competenze [7] — hanno ricevuto attenzione crescente come intervento per il ASD. Una revisione sistematica di Carneiro et al. [7], condotta su 149 studi iniziali e selezionandone 9 secondo criteri PICO e linee guida PRISMA, ha analizzato l'efficacia di serious games progettati per il potenziamento delle competenze sociali in bambini e adolescenti con ASD.

I giochi inclusi nella revisione erano tutti strumenti software sviluppati appositamente, centrati su domini come il riconoscimento delle emozioni, la decodifica delle espressioni facciali, il contatto visivo, la joint attention e la regolazione emotiva. Il cento per cento degli studi analizzati riporta miglioramenti significativi in almeno uno dei costrutti misurati; l'ottantacinque per cento mostra miglioramenti nelle competenze sociali generali, e tutti gli studi riportano effetti positivi sul riconoscimento delle emozioni, sulla prospettiva altrui e sulle abilità comportamentali [7].

La revisione segnala tuttavia alcune limitazioni: campioni ridotti, eterogeneità nei protocolli, rischio di bias elevato nella maggior parte degli studi, e una copertura quasi esclusiva della fascia 7–10 anni. Nessuno degli studi analizzati riguardava implementazioni digitali di giochi da tavolo strutturati.

2.4 Posizionamento del presente lavoro

La letteratura si è concentrata quasi esclusivamente su software sviluppati ex novo, lasciando inesplorato il gioco da tavolo in versione digitale. A differenza dei serious games recensiti da Carneiro et al. [7], progettati per uso individuale o mediato dallo schermo, *La Città degli Imprevisti* mantiene l'interazione faccia a faccia attorno a un tavolo condiviso e richiede la mediazione di un educatore. I domini che esercita — riconoscimento delle emozioni, comunicazione non verbale, prospettiva altrui, collaborazione, gestione della competizione — coincidono con le aree di intervento prioritarie nel ASD identificate dalla letteratura.

Capitolo 3

Tecnologie utilizzate

L'applicazione è sviluppata in TypeScript con il framework Next.js, a sua volta basato su React. Il processo di build produce un insieme di file statici (HTML, JavaScript, CSS) interpretabili da qualsiasi browser moderno, senza necessità di un server applicativo. Gli strumenti adottati sono stati scelti per garantire qualità del codice, efficienza di sviluppo e manutenibilità nel lungo periodo.

3.1 Framework e librerie di sviluppo

3.1.1 Next.js

Next.js è un framework React per applicazioni web che integra nativamente routing basato su file, rendering lato server (SSR) e generazione di siti statici (SSG). È stato scelto come framework principale per la sua capacità di produrre output statici distribuibili su GitHub Pages senza necessità di un server, mantenendo al contempo un'esperienza di sviluppo moderna.

3.1.2 TypeScript

TypeScript è un superset di JavaScript che aggiunge tipizzazione statica al linguaggio, consentendo di individuare errori a tempo di compilazione anziché a runtime. Nel contesto di un progetto con un domain model articolato — composto da classi, manager e interfacce — la tipizzazione ha reso più sicuro il refactoring e ha ridotto significativamente la categoria di bug legati a tipi inattesi.

3.1.3 React

React è la libreria su cui si basa Next.js per la costruzione dell'interfaccia utente tramite componenti dichiarativi. Il modello a componenti consente di strutturare l'interfaccia in unità indipendenti e riutilizzabili, ciascuna responsabile del proprio stato visivo e del proprio ciclo di vita.

3.1.4 Zustand

Zustand è una libreria leggera per la gestione dello stato globale dell'applicazione. A differenza di soluzioni più elaborate come Redux, non richiede *boilerplate* e si integra in modo naturale con TypeScript. Il middleware *persist* consente di sincronizzare automaticamente gli store con il `localStorage` del browser, rendendo lo stato della partita persistente al ricaricamento della pagina senza codice aggiuntivo.

3.1.5 Tailwind CSS

Tailwind CSS è un framework CSS *utility-first*: invece di definire classi semantiche personalizzate, si compone l'aspetto visivo applicando direttamente nel markup classi predefinite di basso livello. Questo approccio ha velocizzato lo sviluppo dell'interfaccia, garantendo coerenza visiva e semplificando la gestione dei layout responsivi.

3.2 Strumenti di qualità del codice

3.2.1 Biome

Biome è uno strumento che unifica in un unico pacchetto le funzionalità di linting, formattazione e organizzazione degli import, sostituendo la tradizionale combinazione ESLint + Prettier con una configurazione singola e tempi di esecuzione notevolmente inferiori. È configurato per essere eseguito automaticamente a ogni commit tramite Husky.

3.2.2 Husky e Commitlint

Husky gestisce i Git hooks, permettendo di eseguire automaticamente controlli prima che un commit venga registrato. Nel progetto è configurato per eseguire Biome (linting e formattazione) e Commitlint (validazione del messaggio di commit) a ogni `git commit`. Commitlint verifica che i messaggi rispettino la convenzione *Conventional Commits*, migliorando la leggibilità della cronologia del progetto.

3.3 Strumenti di testing e documentazione

3.3.1 Vitest

Vitest è un framework di test unitari progettato per integrarsi con l'ecosistema Vite/Next.js. Offre prestazioni superiori rispetto a Jest grazie all'esecuzione nativa dei moduli ES e supporta la misurazione della copertura del codice tramite Istanbul. Viene utilizzato per testare la logica del domain model in isolamento dall'interfaccia.

3.3.2 Playwright

Playwright è un framework per test end-to-end che automatizza browser reali (Chromium, Firefox, WebKit). Permette di verificare il comportamento completo dell'applicazione simulando le interazioni di un utente reale, coprendo scenari che i test unitari non possono rilevare.

3.3.3 Storybook

Storybook è uno strumento per lo sviluppo e la documentazione dei componenti UI in isolamento. Ogni componente è associato a una o più *stories* che ne mostrano i diversi stati e configurazioni, facilitando lo sviluppo visivo indipendente e costituendo una documentazione interattiva dell'interfaccia.

Capitolo 4

Implementazione

4.1 Contesto d’uso e scelte progettuali

L’applicazione è progettata per sessioni di gruppo in presenza, guidate da un educatore. Tutti i partecipanti si trovano nella stessa stanza e condividono un unico dispositivo — tipicamente un tablet o un PC con schermo sufficientemente grande da essere visibile a tutti i giocatori seduti attorno al tavolo. Non vengono utilizzati dadi fisici: il lancio avviene premendo un pulsante sullo schermo, e il risultato viene animato e visualizzato dall’applicazione. Anche l’avanzamento delle pedine è gestito interamente dal software: dopo ogni lancio, la pedina del giocatore attivo si sposta automaticamente lungo il tabellone fino alla casella di destinazione.

La scelta di adottare un dispositivo condiviso anziché assegnare uno schermo a ciascun giocatore è deliberata: preserva la natura faccia a faccia del gioco da tavolo originale, mantiene l’attenzione del gruppo centrata su un unico punto di riferimento e riduce la complessità infrastrutturale. Il ruolo dell’educatore rimane centrale: è lui a facilitare le sfide delle caselle speciali, a valutare le risposte dei giocatori e a registrare l’esito nell’applicazione, che ne applica gli effetti sul tabellone.

4.2 Architettura generale

L’applicazione è una *single-page application* (SPA) prodotta come insieme di file statici (HTML, JavaScript, CSS) tramite la funzionalità di esportazione statica di Next.js. Non richiede un server applicativo: i file vengono distribuiti su GitHub Pages e caricati direttamente dal browser. Tutta la logica di gioco — compresa la gestione dello stato, il salvataggio della partita e la generazione del tabellone — risiede ed è eseguita nel browser dell’utente.



Figura 1: Schermata di gioco: tabellone, barra laterale e dado.

Poiché non vi è comunicazione di rete durante la partita, l'applicazione funziona su qualsiasi dispositivo dotato di un browser moderno: PC, laptop, tablet o qualsiasi altro terminale in grado di eseguire JavaScript. Non è richiesta alcuna installazione.

4.3 Struttura ad alto livello

L'applicazione si articola in due moduli funzionali principali, corrispondenti alle due fasi distinte dell'esperienza d'uso.

Il modulo di **configurazione** permette di definire i parametri della partita e genera il tabellone al termine.

Il modulo di **gioco** gestisce lo svolgimento turno per turno fino a quando un giocatore raggiunge l'ultima casella.

4.4 Specifiche funzionali

Di seguito sono descritte le funzionalità implementate, organizzate per fase d'uso.

Figura 2: Modulo di configurazione: impostazione del numero di giocatori, lunghezza del percorso e tipologie di caselle.

4.4.1 Configurazione della partita

Prima di avviare una partita, il sistema consente di:

- impostare il numero di giocatori (da 2 a 10), ciascuno identificato da un nome e un colore assegnato automaticamente;
- definire il numero di caselle del tabellone (da 10 a 100);
- scegliere quali tipologie di caselle speciali includere tra: Mimo, Quiz, Parole alle spalle, Disegno dettato, Emozioni in musica, Indovina l'emozione, Cosa faresti se..., Test fisico, Caselle movimento;
- regolare la percentuale di caselle speciali rispetto al totale.

La tipologia **Battaglia** non è selezionabile: si attiva automaticamente in caso di collisione tra pedine. Per le **Caselle movimento**, la modalità avanzata consente di specificare il valore esatto di avanzamento o retrocessione per ogni singola casella.

La configurazione viene persistita in locale tramite il middleware `persist` di Zustand, così da essere disponibile anche in caso di ricaricamento della pagina.

4.4.2 Svolgimento del turno

Il turno di ogni giocatore segue la sequenza:

1. Il giocatore attivo lancia il dado (valore casuale 1–6).
2. La pedina si sposta lungo il tabellone del numero di caselle corrispondente al risultato del lancio.
3. Se la casella di arrivo è normale, il turno termina e passa al giocatore successivo.
4. Se la casella è speciale, viene mostrata una finestra modale con la sfida corrispondente.
5. L'educatore valuta l'esito e registra il risultato; il sistema applica le conseguenze (avanzamento, retrocessione o salto del turno).
6. In caso di collisione tra due pedine sulla stessa casella, si attiva automaticamente la meccanica di Battaglia.

4.4.3 Salvataggio e ripristino della partita

Lo stato completo della partita (posizioni delle pedine, turno corrente, stato dei mazzi) è serializzabile in formato JSON ed esportabile come file. Una partita interrotta può essere ripristinata caricando il file salvato dalla schermata dedicata.

4.5 Descrizione funzionale

L'applicazione si articola in sei schermate principali. La navigazione avviene tramite un header persistente che espone due elementi: un link diretto alla schermata delle istruzioni e un pulsante Menu che apre una finestra modale con i collegamenti a Nuova partita, Continua partita, Carica partita e Feedback.

4.5.1 Schermata Home — Configurazione della partita

La schermata iniziale permette di configurare una nuova partita: numero e nomi dei giocatori, lunghezza del percorso, tipologie di caselle speciali da includere e relativa percentuale di distribuzione.

È disponibile anche una **modalità avanzata** che consente di definire manualmente la disposizione delle caselle (descritta nella sezione successiva).

Una volta completata la configurazione, il sistema genera il tabellone e avvia la partita.

4.5.2 Schermata di gioco

È la schermata principale, composta da tre aree:

- **Tabellone** — Griglia di caselle numerate, identificate visivamente per tipologia; le pedine si animano al momento dello spostamento.
- **Barra laterale** — Mostra il giocatore attivo, il numero del round e le posizioni di tutti i giocatori.
- **Dado e modali** — Un pulsante permette al giocatore attivo di lanciare il dado; se la casella di arrivo è speciale, si apre la finestra modale corrispondente alla sfida.

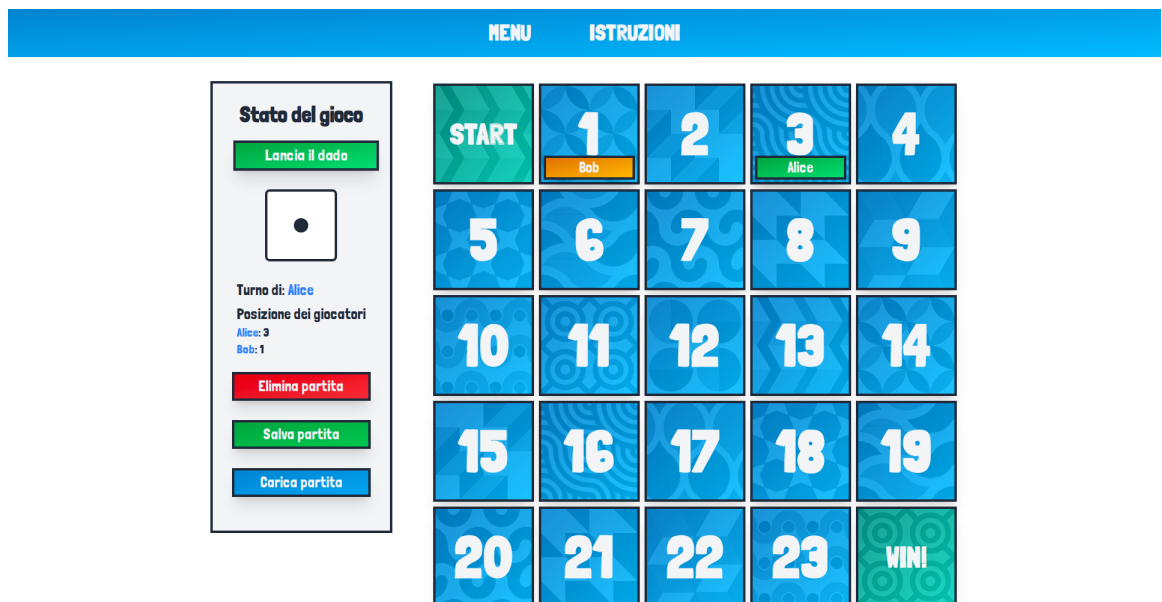
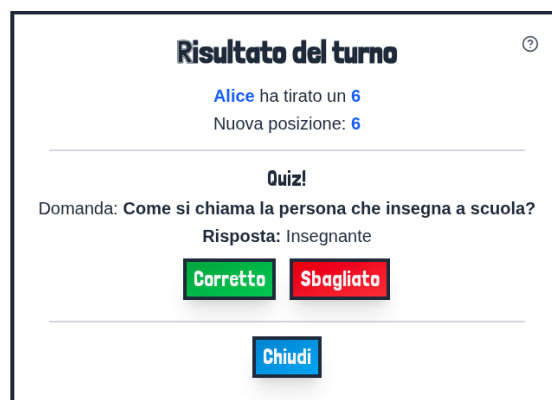


Figura 3: Animazione del lancio del dado durante il turno del giocatore attivo.

Le sfide attivabili sono le dieci descritte nella sezione 1.5: Mimo, Quiz, Parole alle spalle, Disegno dettato, Emozioni in musica, Indovina l'emozione, Cosa faresti se..., Test fisico, Battaglia e Caselle movimento. Ciascuna viene presentata attraverso una finestra modale sovrapposta al tabellone al momento dell'attivazione, come mostrato nelle figure 4 e 5.



Mimo.



Quiz.



Parole alle spalle.



Disegno dettato.

Figura 4: Finestre modali delle sfide (1/2): Mimo, Quiz, Parole alle spalle, Disegno dettato.



Emozioni in musica.



Indovina l'emozione.



Cosa faresti se...



Test fisico.

Figura 5: Finestre modali delle sfide (2/2): Emozioni in musica, Indovina l'emozione, Cosa faresti se..., Test fisico.

4.5.3 Menu di navigazione

Il menu è accessibile in qualsiasi momento tramite il pulsante **MENU** nell'header, indipendentemente dalla schermata corrente. L'apertura sovrappone una finestra modale al contenuto sottostante senza interrompere lo stato della partita.

Il menu espone quattro collegamenti:

- **Continua partita** — reindirizza alla schermata di gioco, riprendendo la sessione in corso.
- **Nuova partita** — reindirizza alla schermata Home per configurare una partita da zero.
- **Carica partita** — reindirizza alla schermata Ripristina partita.
- **Dai un feedback!** — reindirizza alla schermata Feedback.

Una sezione dedicata alle impostazioni audio consente di abilitare o disabilitare gli effetti sonori dell'applicazione tramite una casella di spunta; la modifica è immediata e persistente per tutta la sessione.

4.5.4 Schermata Istruzioni

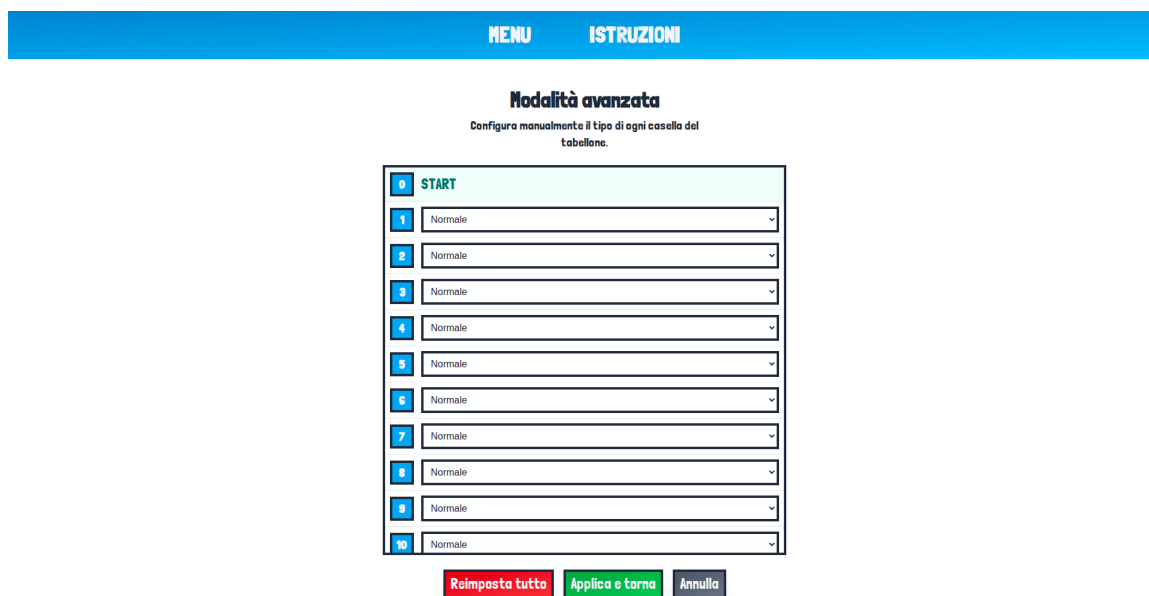
Presenta le regole del gioco strutturate in sezioni (introduzione, configurazione, svolgimento del turno, caselle speciali, fine della partita), navigabili tramite un indice interno con link di ancoraggio. Ogni sezione è corredata da video dimostrativi che mostrano l'applicazione in funzione nei principali scenari di gioco. La schermata è raggiungibile direttamente dall'header in qualsiasi momento, senza perdere lo stato della partita in corso.

4.5.5 Schermata Modalità avanzata

Consente di assegnare manualmente il tipo a ogni singola casella del tabellone, sostituendo la distribuzione casuale. L'interfaccia presenta un elenco scorrevole di tutte le caselle: ciascuna è identificata da un badge numerico il cui colore riflette il tipo attualmente assegnato. La prima e l'ultima casella (Start e Fine) sono bloccate e non modificabili. Per ogni casella intermedia è disponibile un menu a tendina con tutte le tipologie disponibili; selezionando il tipo *movimento* compare un campo numerico aggiuntivo per specificare il numero di caselle di avanzamento o retrocessione (da -5 a +5). I pulsanti in fondo alla schermata permettono di azzerare la configurazione, applicarla tornando alla schermata principale o annullare le modifiche.



Figura 6: Menu di navigazione: collegamenti alle schermate principali e controllo audio.



Modalità avanzata
Configura manualmente il tipo di ogni casella del tabellone.

START

- 1 Normale
- 2 Normale
- 3 Normale
- 4 Normale
- 5 Normale
- 6 Normale
- 7 Normale
- 8 Normale
- 9 Normale
- 10 Normale

Reimposta tutto **Applica e torna** **Annulla**

Figura 7: Schermata modalità avanzata: assegnazione manuale del tipo a ogni casella del tabellone.

4.5.6 Schermata Ripristina partita

Consente di riprendere una sessione interrotta attraverso due modalità distinte. La prima permette di caricare un file JSON esportato in precedenza: l'utente seleziona il file dal proprio dispositivo e avvia il ripristino; il sistema valida il contenuto e, in caso di file non valido, mostra un messaggio di errore. La seconda modalità è disponibile quando il browser ha conservato automaticamente una partita nel `localStorage`: in tal caso compare un pulsante aggiuntivo per riprendere la sessione senza bisogno di alcun file. In entrambi i casi, al termine del ripristino l'utente viene reindirizzato direttamente alla schermata di gioco.

4.5.7 Schermata Feedback

Contiene un modulo strutturato per raccogliere valutazioni sull'applicazione da parte di docenti e alunni del CFPIL. Il modulo richiede il nome del compilatore, il ruolo (docente, alunno o altro) e il tipo di esperienza maturata con il gioco (versione fisica, digitale o entrambe). Una sezione di valutazione numerica (scala 1–5) raccoglie giudizi su accessibilità, fedeltà della versione digitale rispetto a quella fisica, chiarezza visiva e comfort sonoro. Tre campi di testo libero permettono infine di descrivere gli aspetti positivi, le

difficoltà riscontrate e i suggerimenti per miglioramenti futuri. Le risposte sono inviate a un servizio esterno e archiviate per analisi successive.

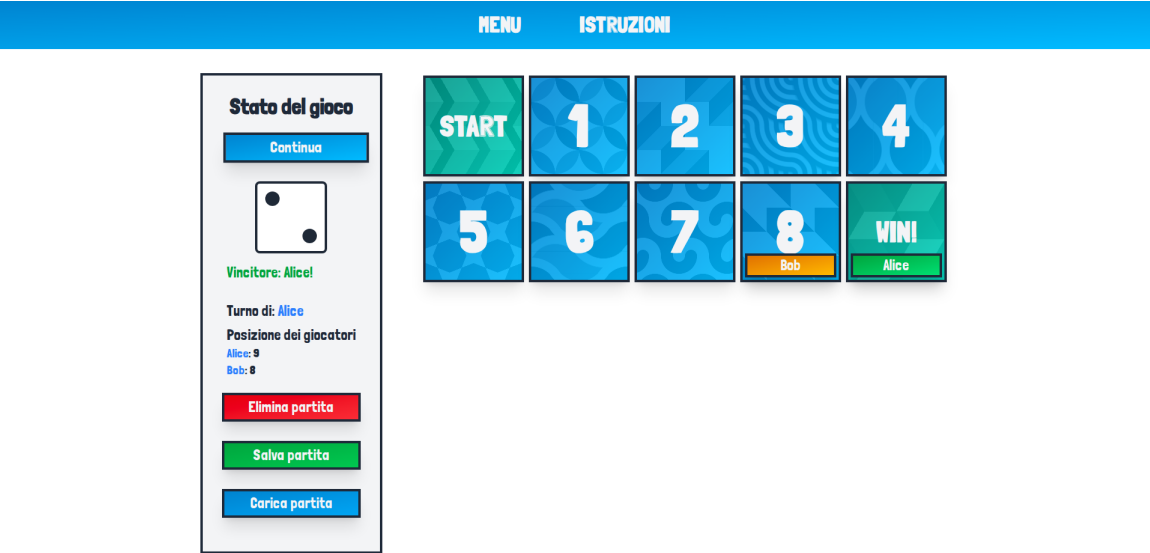


Figura 8: Schermata di fine partita con classifica finale dei giocatori.

4.6 Architettura dei moduli

L'applicazione segue un'architettura a tre strati con separazione netta tra logica di dominio, gestione dello stato e presentazione:

Strato	Modulo
Presentation Layer	Componenti React (src/app/)
State Management Layer	Store Zustand (src/store/)
Domain Model Layer	Logica di gioco (src/model/)

Tabella 1: Architettura a strati dell'applicazione.

Questa separazione consente di testare la logica di gioco indipendentemente dall'interfaccia e di aggiornare i componenti visivi senza modificare le regole di gioco.

4.6.1 Domain Model

Il domain model contiene tutta la logica di gioco, completamente slegata da React e dal browser. La classe `Game` è l'orchestratore principale: mantiene i riferimenti a tutti i sottocomponenti (tabellone, giocatori, dado, mazzi, manager) ed espone i metodi pubblici che il layer di stato chiama per far avanzare la partita. Il metodo `playTurn()` esegue un turno completo — lancio del dado, spostamento della pedina, elaborazione della casella — e restituisce il tipo di azione da mostrare all'utente. I metodi `toJSON()` e `fromJSON()` gestiscono la serializzazione dello stato per il salvataggio su file.

Ogni aspetto della partita è delegato a un manager specializzato:

- `TurnManager` — traccia il turno corrente, il round e il giocatore attivo;
- `MovementManager` — sposta le pedine, gestisce bonus/malus di movimento e collisioni;
- `GameStateManager` — verifica la condizione di vittoria;
- `SpecialSquareProcessor` — determina quale azione speciale si attiva su una casella;
- `BattleManager` e i manager di ogni tipologia di sfida (es. `MimeManager`, `QuizManager`) — applicano gli effetti dell'esito di ciascuna sfida.

Le caselle del tabellone sono modellate come classi distinte (es. `MimeSquare`, `QuizSquare`) che estendono la classe base `Square` e sono ricostruite tramite il pattern `Builder` al momento del ripristino di una partita salvata.

Ogni tipologia di casella speciale dispone di un mazzo di carte dedicato (es. `MimeDeck`, `QuizDeck`), mescolato all'inizio della partita e consumato in sequenza per garantire varietà e non ripetizione dei contenuti nella stessa sessione.

4.6.2 State Management

Lo stato globale è gestito con `Zustand`, suddiviso in tre store indipendenti persistiti in `localStorage`:

- `game-store` — stato della partita in corso: istanza serializzata di `Game`, risultato del dado, tipo di azione corrente, stato delle finestre modali;
- `config-store` — configurazione della partita: nomi e numero dei giocatori, numero di caselle, tipologie e percentuale di caselle speciali;
- `sound-store` — preferenza dell'utente riguardo agli effetti sonori.

4.6.3 Presentation Layer

Il presentation layer comprende sia le pagine che i componenti React. In Next.js le pagine (`page.tsx`) sono esse stesse componenti: la distinzione è strutturale — determinano il routing — ma non concettuale.

Le pagine dell'applicazione sono:

- `app/page.tsx` — schermata di configurazione della partita;
- `app/game/page.tsx` — schermata principale di gioco;
- `app/instructions/page.tsx` — istruzioni di gioco con video dimostrativi;
- `app/advanced-mode/page.tsx` — configurazione avanzata del tabellone;
- `app/restore-game/page.tsx` — ripristino partita da file;
- `app/feedback/page.tsx` — modulo di feedback.

I componenti principali dell'interfaccia sono:

- `board.tsx` — tabellone come griglia CSS con numero di colonne variabile in base alla dimensione;
- `square.tsx` — singola casella con numero, tipo visivo e pedine presenti;
- `pawn.tsx` — pedina con colore assegnato e animazione al momento dello spostamento;
- `dice.tsx` — dado con animazione al momento del lancio;
- `left-bar.tsx` — barra laterale con turno attivo, round corrente e classifica;
- `turn-result-modal.tsx` — finestra modale delegata a un sottocomponente specifico per ogni tipo di sfida.

I componenti primitivi riutilizzabili (`Button`, `Input`, `Label`, `Select`) sono sviluppati e documentati in isolamento tramite Storybook.

4.7 Scelte implementative rilevanti

4.7.1 Il tipo `GameActionResult` e il `Command Pattern`

La comunicazione tra il domain model e il layer di stato avviene tramite il tipo `GameActionResult`, un oggetto TypeScript che `playTurn()` restituisce al termine di ogni turno. Questa scelta riflette il *Command Pattern*: invece di eseguire direttamente un'azione sull'interfaccia, il domain model produce un oggetto che *descrive* l'azione da compiere, lasciando al chiamante la responsabilità di decidere come gestirla.

Il campo `type` è un'unione discriminata di stringhe letterali — una per ogni tipologia di sfida più il valore "none" per i turni senza azione speciale. Il campo `data` trasporta i dati specifici dell'azione (ad esempio la carta estratta o i giocatori coinvolti in una battaglia). Lo store legge `type` per determinare quale finestra modale aprire, e `data` per popolarla con il contenuto corretto, senza che il domain model abbia alcuna dipendenza dall'interfaccia. Il ciclo si chiude quando l'educatore registra l'esito della sfida: lo store chiama il corrispondente metodo `resolve*`() su `Game`, che applica le conseguenze sul tabellone e aggiorna lo stato della partita.

Listing 4.1: Tipo di ritorno di `playTurn()` (`src/model/managers/types.ts`)

```

1 export type GameActionResult = {
2   type:
3     | "battle" | "mime"      | "quiz"
4     | "backwrite"           | "face-emotion"
5     | "music-emotion"       | "physical-test"
6     | "what-would-you-do"   | "dictation-draw"
7     | "none";
8   data?: Battle | Mime | Quiz | BackWrite | FaceEmotion
9         | MusicEmotion | PhysicalTest | WhatWouldYouDo
10        | DictationDraw;
11   diceResult: number;
12   actionType: string | null;
13 };

```

Il metodo `playTurn()` in `Game` sfrutta questo tipo per incapsulare l'intera logica del turno — gestione del turno saltato, lancio del dado, movimento, rilevamento collisioni ed elaborazione della casella speciale — restituendo un valore strutturato che il layer di stato consuma senza dover conoscere i dettagli interni.

Listing 4.2: Metodo `playTurn()` (`src/model/game.ts`)

```

1 playTurn(): GameActionResult {
2   const currentPlayer = this.turnManager.getCurrentPlayer();
3

```

```
4   if (currentPlayer.mustSkipTurn()) {  
5       this.turnManager.advanceTurn();  
6       return { type: "none", diceResult: 0, actionType: null };  
7   }  
8  
9   const diceValue = this.dice.roll();  
10  const result = this.executePlayerTurn(currentPlayer,  
11      diceValue);  
12  this.turnManager.advanceTurn();  
13  
14  return { ...result, diceResult: diceValue };  
15  }
```

4.7.2 Gestione dello stato con Zustand e persistenza

Lo store `useGameStore` è il punto di raccordo tra il domain model e l'interfaccia utente. È definito tramite `create()` di Zustand, avvolto nel middleware `persist`, e si articola in due sezioni distinte: lo *stato* e le *azioni*.

La sezione di stato contiene due categorie di campi: quelli che riflettono la partita in corso (`game`, istanza della classe `Game`, e `gameData`, la sua rappresentazione JSON) e quelli che descrivono la situazione dell'interfaccia (`isModalOpen`, `isDiceModalOpen`, `diceResult`, `actionType`, `actionData` e altri).

Le azioni sono funzioni che i componenti React chiamano in risposta agli eventi dell'utente. La più importante è `playTurn()`: recupera l'istanza di `Game` dallo store, invoca il metodo omonimo del domain model, riceve il `GameActionResult` e aggiorna lo stato dell'interfaccia di conseguenza — impostando il risultato del dado, il tipo di azione attiva e aprendo la finestra modale appropriata. Analogamente, ciascuna azione `resolve*`() delega la risoluzione al metodo corrispondente di `Game` e, al termine, aggiorna la serializzazione JSON della partita tramite `toJSON()`.

Il middleware `persist` sincronizza automaticamente lo store con il `localStorage` del browser. L'opzione `partialize` limita la persistenza al solo campo `gameData`: in questo modo lo stato della partita sopravvive al ricaricamento della pagina, mentre i campi transitori dell'interfaccia vengono reinizializzati ai valori di default.

Listing 4.3: Store di gioco con persistenza selettiva (`src/store/game-store.ts`)

```
1  export const useGameStore = create<GameStore>()(  
2      persist(  
3          (set, get) => ({  
4              game: null,  
5              gameData: null,  
6              isModalOpen: false,
```

```

7      // ...altri campi UI...
8
9      actions: {
10         playTurn: () => {
11             const { game } = get();
12             if (!game) return;
13
14             const { diceResult, actionType, data } =
15                 game.playTurn();
16             // aggiorna lo stato UI in base al risultato...
17         },
18         // ...altri metodi...
19     },
20     {
21         name: "game-store",
22         partialize: (state) => ({
23             gameData: state.gameData,
24             hasSavedGame: !!state.gameData,
25         }),
26     },
27 ),
28 );

```

4.8 Design pattern utilizzati

La progettazione dell'applicazione ha fatto uso di diversi design pattern consolidati [8], applicati in modo coerente con le responsabilità dei moduli descritti nelle sezioni precedenti.

4.8.1 Command Pattern

Il pattern Command è applicato nella comunicazione tra il domain model e il layer di stato tramite il tipo `GameActionResult` (si veda la sezione 4.7.1): ogni turno produce un oggetto che *describe* l'azione da eseguire, disaccoppiando il mittente (il domain model) dal ricevente (lo store e l'interfaccia).

4.8.2 Facade Pattern

La classe `Game` agisce da *Facade*: espone un'interfaccia semplice e coerente (`playTurn()`, `resolve*()`, `toJSON()`) nascondendo la complessità interna dei manager, dei mazzi e

del tabellone. I componenti React e lo store non interagiscono mai direttamente con i sottosistemi interni.

4.8.3 Builder Pattern

Il modulo `square-builder.ts` implementa il pattern Builder per ricostruire le istanze delle caselle a partire dalla loro rappresentazione JSON durante il ripristino di una partita salvata. Poiché ogni tipologia di casella è una classe distinta, il Builder centralizza la logica di selezione del costruttore corretto in base al campo `type`.

4.8.4 Observer Pattern

Zustand adotta implicitamente il pattern Observer: i componenti React si *sottoscrivono* a porzioni specifiche dello store tramite selettori, e vengono ri-renderizzati automaticamente solo quando i campi osservati cambiano. Questo evita aggiornamenti superflui dell'interfaccia e mantiene i componenti disaccoppiati dalla logica di aggiornamento dello stato.

4.8.5 Manager Pattern

La classe `Game` non implementa da sola tutta la logica di gioco, ma delega ciascuna responsabilità a un manager specializzato: `TurnManager` per la gestione dei turni, `MovementManager` per gli spostamenti e le collisioni, `GameStateManager` per la verifica della vittoria, `SpecialSquareProcessor` per l'attivazione delle sfide, e un manager dedicato per ogni tipologia di sfida (es. `MimeManager`, `QuizManager`). Questo approccio — talvolta chiamato *Manager Pattern* o *Specialist Pattern* — consente di mantenere la classe `Game` snella e leggibile, concentrando ciascuna regola di gioco nel modulo che ne è responsabile, con benefici diretti sulla testabilità e sulla manutenibilità del codice.

4.8.6 Model-View-Controller

L'architettura a tre strati dell'applicazione rispecchia il pattern architetturale *Model-View-Controller* (MVC): il Domain Model (`src/model/`) corrisponde al *Model*, i componenti React (`src/app/`) alla *View* e gli store Zustand (`src/store/`) al *Controller*, che media le interazioni dell'utente verso il modello e propaga gli aggiornamenti di stato verso l'interfaccia.

4.9 Modalità multi-dispositivo

La modalità di base dell'applicazione prevede un unico dispositivo condiviso tra tutti i partecipanti. Una seconda modalità, denominata *multi-dispositivo*, estende il gioco consentendo a ogni giocatore di interagire attraverso il proprio smartphone: il lancio del dado avviene sullo schermo personale e le sfide delle caselle speciali vengono distribuite tra i dispositivi in modo da preservare, ove pertinente, l'elemento di segretezza — ad esempio, nel Mimo o nel Disegno dettato, il giocatore attivo vede il tema da rappresentare sul proprio schermo prima che gli altri lo scoprano.

La modalità è accessibile dalla schermata di configurazione tramite la voce **Multi-dispositivo**. Una volta selezionata, il flusso di gioco si articola in una fase di *lobby*, in cui i partecipanti si connettono alla sessione, seguita dall'avvio della partita da parte dell'educatore.

4.9.1 Architettura delle sessioni

La modalità multi-dispositivo richiede una componente *server-side* per sincronizzare lo stato della partita tra i dispositivi dei giocatori. A tal fine, l'applicazione fa uso delle *API Routes* di Next.js, che consentono di definire endpoint HTTP direttamente nel progetto senza la necessità di un server separato.

Lo stato di sessione è rappresentato da un oggetto `SessionState` che include l'elenco dei giocatori connessi, la fase corrente della partita, il turno attivo e gli esiti delle sfide. La persistenza di questo stato è gestita tramite un meccanismo a tre livelli, scelto in cascata in base all'ambiente di esecuzione:

1. **Vercel KV** — in produzione, lo stato è memorizzato su Vercel KV, un servizio di *key-value store* basato su Redis ospitato sulla piattaforma Vercel;
2. **ioredis** — in sviluppo locale con Redis disponibile, si utilizza il client `ioredis` per connettersi a un'istanza Redis locale;
3. **Map in memoria** — in assenza di Redis, lo stato è mantenuto in una `Map` JavaScript in memoria del processo `Node.js`, sufficiente per lo sviluppo locale su singola istanza.

Questa struttura garantisce portabilità tra ambienti senza modifiche al codice applicativo: la selezione del livello appropriato avviene in fase di inizializzazione in base alle variabili di ambiente disponibili.

La creazione di una sessione avviene tramite una richiesta `POST` all'endpoint `/api/sessions`, che restituisce un identificatore di sessione (`sessionId`) e un `hostToken`, un token opaco conservato nel `localStorage` del dispositivo dell'educatore e allegato alle successive richieste privilegiate per autorizzarle.

4.9.2 Connessione dei giocatori tramite QR code

Una volta creata la sessione, l'applicazione genera un codice QR che i giocatori inquadrano con il proprio smartphone. Il codice QR contiene l'URL della pagina di accesso, che include il `sessionId` come parametro. Il giocatore inserisce il proprio nome e invia una richiesta POST all'endpoint `/api/sessions/[id]/join`, che registra il nuovo partecipante nella sessione e restituisce un `playerId` univoco.

Il giocatore viene quindi reindirizzato alla propria schermata di gioco, raggiungibile all'URL `/player/[sessionId]/[playerId]`. L'identità del giocatore è interamente codificata nell'URL: anche in caso di chiusura accidentale della scheda o di ricaricamento della pagina, il giocatore recupera il proprio stato semplicemente riaprendo lo stesso indirizzo.

4.9.3 Comunicazione in tempo reale tramite *long polling*

Per notificare i dispositivi dei giocatori degli aggiornamenti di stato senza l'uso di connessioni persistenti, la modalità multi-dispositivo adotta la tecnica del *long polling*. Ogni dispositivo invia periodicamente una richiesta GET all'endpoint `/api/sessions/[id]`, passando come parametro il valore `updatedAt` dell'ultimo stato ricevuto. Il server, anziché rispondere immediatamente, mantiene la connessione aperta per un massimo di otto secondi, interrogando lo *store* ogni trecento millisecondi; restituisce la risposta non appena rileva che il campo `updatedAt` dello stato corrente è più recente di quello del client, oppure allo scadere del *timeout*.

Questo approccio riduce il numero di invocazioni delle *serverless function* di circa cinque volte rispetto al *polling* a intervallo fisso, un aspetto rilevante ai fini della compatibilità con il piano gratuito di Vercel, che impone limiti al numero di esecuzioni mensili. La scelta del *long polling* in luogo dei WebSocket è motivata dalla stessa considerazione: il piano *Hobby* di Vercel non supporta connessioni WebSocket persistenti, mentre le richieste HTTP di lunga durata sono gestite nativamente dalle *serverless function*.

4.9.4 Rilevamento dell'inattività

Per evitare il consumo inutile di risorse in caso di dispositivi lasciati incustoditi, la modalità multi-dispositivo include un meccanismo di rilevamento dell'inattività. Il *polling* viene sospeso automaticamente dopo trenta minuti di inattività del giocatore; sullo schermo compare un *overlay* con il messaggio “*Stai ancora giocando?*”, che invita l'utente a confermare la propria presenza. In assenza di interazione per ulteriori cinque minuti, il *polling* si interrompe definitivamente. Qualsiasi interazione con il dispositivo — un tocco sullo schermo, un'azione di gioco — è sufficiente a riprendere il ciclo di aggiornamento.

4.9.5 Svolgimento del turno in modalità multi-dispositivo

Nel corso della partita, il flusso di ogni turno si svolge come segue:

1. L'educatore avvia la partita premendo il pulsante **Avvia partita** dalla schermata di *lobby*; tutti i dispositivi ricevono l'aggiornamento di stato e passano alla schermata di gioco.
2. Quando è il proprio turno, il giocatore attivo lancia il dado dal proprio dispositivo; il risultato viene comunicato al server e propagato agli altri partecipanti.
3. Se la casella di arrivo non attiva alcuna sfida speciale, il turno termina e il controllo passa al giocatore successivo.
4. Se la casella attiva una sfida, la gestione varia in base alla tipologia:
 - **Quiz** — il testo della domanda è visibile a tutti; l'educatore mostra la risposta corretta premendo il pulsante **Mostra risposta** e registra l'esito (successo o insuccesso);
 - **Mimo e Disegno dettato** — il tema da rappresentare compare esclusivamente sul dispositivo del giocatore attivo; gli altri vedono solo il tipo di sfida. Al termine, l'educatore seleziona quale giocatore ha indovinato, e quest'ultimo riceve il beneficio;
 - **Parole alle spalle** — la parola da tracciare compare sul dispositivo del giocatore attivo; il vincitore è automaticamente il compagno designato come destinatario della comunicazione, senza necessità di selezione aggiuntiva;
 - **Emozione facciale, Emozione musicale, Test fisico, Cosa faresti se...** — la sfida è visibile a tutti; l'educatore registra l'esito al termine;
 - **Battaglia** — i due giocatori coinvolti effettuano la sfida sui rispettivi dispositivi; il server determina l'esito e aggiorna le posizioni.
5. Al termine di ogni sfida, l'educatore conferma l'esito sull'interfaccia; il server aggiorna lo stato della sessione e tutti i dispositivi ricevono la notifica dell'avanzamento.

Al termine della partita, tutti i dispositivi visualizzano la schermata di fine gioco con il vincitore e la classifica finale.

Capitolo 5

Test

5.1 Raccolta di feedback

Al termine delle sessioni di utilizzo, i partecipanti del CFPIL di Varese sono stati invitati a compilare il modulo di feedback integrato nell'applicazione. Il modulo raccoglie:

- **nome;**
- **ruolo** — docente, alunno o altro;
- **esperienza pregressa con il gioco** — versione fisica, digitale o entrambe;
- **accessibilità** — valutazione in scala 1-5;
- **fedeltà della versione digitale rispetto a quella fisica** — valutazione in scala 1-5;
- **chiarezza visiva** — valutazione in scala 1-5;
- **comfort sonoro** — valutazione in scala 1-5;
- **aspetti positivi;**
- **difficoltà riscontrate;**
- **suggerimenti di miglioramento.**

Capitolo 6

Conclusioni

Bibliografia

- [1] Luca Surian. *L'autismo - Conoscerlo e affrontarlo*. Il Mulino, 2021.
- [2] CFPIL sito ufficiale. <https://agenziaformativa.va.it/sedi/varese-cfpil/>, 2026.
- [3] Gary B. Mesibov, Victoria Shea, and Eric Schopler. *The TEACCH Approach to Autism Spectrum Disorders*. Springer, 2005.
- [4] Orla Walsh, Conor Linehan, and Christian Ryan. Is there evidence that playing games promotes social skills training for autistic children and youth? *Autism*, 29(2):329–343, 2024.
- [5] Gray Atherton and Liam Cross. The use of analog and digital games for autism interventions. *Frontiers in Psychology*, 12:669734, 2021.
- [6] Gabriele Mari. Gioco strutturato, gioco modificato: un'esperienza di giochi da tavolo con bambini e ragazzi con autismo. *L'integrazione scolastica e sociale*, 17(4):315–321, 2018.
- [7] Ana Carneiro, Tânia Correia, Sara Felizardo, and Maria da Graça Ferreira. Serious games for developing social skills in children and adolescents with autism spectrum disorder: A systematic review. *Healthcare*, 12(5):508, 2024.
- [8] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.